Spring

Spring

开源的轻量级 Java 开发框架

核心功能:

- **依赖注入（DI）**：在容器中建立**bean与bean之间的依赖关系**，降低代码耦合度。
- **面向切面编程（AOP）**：支持定义切面，方便实现日志记录、权限控制、事务管理等横切关注点功能。
- **数据访问与事务管理**：整合了 JDBC、JPA、Hibernate 等持久层框架，简化数据库操作，并支持声明式事务管理。

Spring 提供的核心功能主要是 **IoC 和 AOP**

框架	定位与核心功能	特点
Spring	轻量级容器框架，核心是 **IoC（控制反转）** 和 **AOP（面向切面编程）**	提供基础 Bean 管理和扩展能力，支持模块化开发（如 JDBC、事务等）。
SpringMVC	基于 Servlet 的 **Web 框架**，属于 Spring 的子模块	实现 MVC 分层架构，处理 HTTP 请求、路由、视图解析等（如 `@Controller`）。
SpringBoot	**简化 Spring 应用开发**的脚手架框架	内嵌 Tomcat/Jetty、自动配置（`@EnableAutoConfiguration`）、Starter 依赖（开箱即用）。
SpringCloud	**微服务架构解决方案**，基于 SpringBoot 的扩展	提供分布式系统工具集（如服务注册发现 Eureka、配置中心 Config、网关 Zuul 等）。

总结:

- Spring 是基础框架，SpringMVC 是其 Web 模块，SpringBoot 是快速开发工具，SpringCloud 是微服务全家桶。
- SpringBoot 简化了 Spring 的配置，而 SpringCloud 在 SpringBoot 基础上构建分布式系统

小红书
小红书号:49607377662

SpringBoot

Spring Boot 是一个基于 **Spring 框架** 的快速开发工具，传统的Spring需要手动配置数据源、事务管理器等，Spring Boot通过**自动配置、起步依赖、内嵌服务器**等特性，简化了spring应用程序的搭建配置。

SpringBoot启动过程

1. **启动`main`方法**：调用`SpringApplication.run(...)`。
2. **创建SpringApplication实例**：初始化应用程序上下文。
3. **运行`run`方法**：
   - 创建并配置**应用上下文环境（Environment）**。
   - 创建**应用上下文（ApplicationContext）**。
   - **刷新应用上下文**：这是最核心的一步`refresh()`，它：
     a. 加载所有**Bean定义**（通过`@ComponentScan`等）。
     b. 执行**Bean工厂后处理器**（如`@Configuration`类的解析）。
     c. **实例化所有单例Bean**（非懒加载的）。
     d. 发布**应用程序启动完成事件**。
4. 启动内嵌的**Web容器**（如Tomcat）。
5. 应用启动完成，等待请求。

自动配置

根据**项目的依赖（如类路径中的库）自动配置 Spring 应用**。

- **自动配置原理：**
  - **条件化配置**：通过 `@Conditional` 系列注解动态判断是否启用某个配置类。
  - **依赖触发**：根据项目的依赖（如 `spring-boot-starter-web`）自动引入相关配置。
  - **默认值优化**：基于约定优于配置的原则，提供合理的默认值（如内嵌 Tomcat 服务器）。
- **自动配置的工作流程**
  - 根据类路径中的依赖自动配置 Bean，无需手动编写 XML 或 JavaConfig。
    - 如检测到`DataSource`类则自动配置数据库连接
  - 通过 `@EnableAutoConfiguration` 激活，加载 `META-INF/spring.factories` 中注册的配置类。

小红书
小红书号：49607377662

- **条件化注解**（如 `@ConditionalOnClass`）确保仅在满足条件时启用配置（如类路径存在特定库）。

> **示例**：
> 引入 `spring-boot-starter-web` 后，自动配置 Tomcat 服务器和 Spring MVC 组件（如 `DispatcherServlet`）

起步依赖

提供预定义的依赖集合，解决依赖管理和版本冲突问题。

- 提供了很多开箱即用的功能和快速启动的依赖（Starter）,可以通过添加相应的 Starter 依赖来快速集成各种功能，如 Web 开发、数据库访问、安全认证等
  - `spring-boot-starter-web`：Web 开发（RESTful API）。
  - `spring-boot-starter-test`：单元测试（JUnit、Mockito）。
  - `spring-boot-starter-security`：安全认证。
- **自定义 Starter**：
  1. 创建一个模块，定义 `META-INF/spring.factories` 并注册自动配置类。
  2. 在项目中引入该 Starter 即可复用配置。

内嵌web服务器

默认集成 Tomcat，可以将应用打包成可执行的 JAR 文件，直接通过 `java -jar` 命令运行

> Spring 应用通常需要部署到外部的应用服务器（如 Tomcat、Jetty 等）中，部署过程相对复杂，需要将应用打包成 WAR 文件，然后部署到应用服务器中，还需要对应用服务器进行相应的配置。

Tomcat IO模式 & 其他IO模式

- **BIO (Blocking IO)**：早期Tomcat版本默认。一个请求需要一个线程处理，并发高性能差。
- **NIO (Non-blocking IO)**：**Tomcat 8以后默认模式。**使用Selector多路复用器，可以用一个或少量线程处理大量连接，高并发性能好。
- **AIO (Asynchronous IO)**：异步IO，基于回调。在Linux上性能提升不明显，Tomcat支持但默认不采用。
- **APR (Apache Portable Runtime)**：使用JNI调用本地库（如Apache的APR库），主要针对静态资源处理性能有提升。

SpringMVC

Spring MVC 是 Spring 框架中的一个模块，用于构建基于 Web 的应用程序。

遵循**模型-视图-控制器（Model-View-Controller，MVC）**的设计模式:

- 模型用于存储和处理数据，视图用于展示数据，而控制器则处理请求并决定响应的行为。

使用Spring MVC可以将业务逻辑与表现层（前端）进行分离，从而更容易开发、测试和维护 Web 应用。。

小红书
小红书号：49607377662

Spring MVC 的工作流程

1. 用户通过浏览器发送请求。
2. 请求到达 `DispatcherServlet`(Spring MVC的**核心中央处理器**)。
3. `DispatcherServlet` 根据请求 URL 查找合适的控制器（通过 `HandlerMapping` **处理器映射器**）。
4. 控制器处理请求，执行业务逻辑，生成模型数据和视图名。
5. `DispatcherServlet` 使用 `ViewResolver` （视图解析器）来 解析视图名，找到实际视图。
6. `DispatcherServlet` 将Model对象传递给视图对象，视图展示数据生成最终的HTML响应。
7. 最终生成的HTML响应通过HTTP响应发送回客户端（浏览器）。

MyBatis

MyBatis-基于java的持久层框架，用来**简化 JDBC 操作数据库的过程**，你只需要写 SQL 和 XML/注解，MyBatis 会自动帮你把 Java 对象和数据库的记录映射起来。

通过映射文件或者注解，将 SQL 语句与 Java 对象进行绑定。

MyBatis中接口绑定有几种实现方式,是怎么实现的?

- 通过注解绑定，在接口的方法上面加上 @Select@Update等注解里面包含Sql语句来绑定(Sql语句比较简单的 时候，推荐注解绑定)
- 通过xml里面写SQL来绑定, 指定xml映射文件里面的namespace必须为接口的全路径名(SQL语句比较复杂的 时候，推荐xml绑定)

Spring IoC

IoC（Inversion of Control:控制反转） 是一种设计思想，而不是一个具体的技术实现。IoC通过**依赖注入**将对象的创建和管理交给容器

使用对象时由**主动new去产生对象**转换为**由外部提供对象**，将对象的创建和管理的控制权从应用程序代码中反转到**IoC 容器**中。

控制反转：对象的创建和依赖关系的管理从代码中分开，交给 Spring 容器统一管理。

- **控制**：指的是对象创建（实例化、管理）的权力
- **反转**：控制权交给外部环境（IoC 容器）

小红书
小红书号:49607377662

依赖注入（DI）

依赖注入（DI）是实现控制反转（IoC）思想最主要的技术手段。

其核心思想是**将对象的依赖关系由外部容器（如 Spring IoC 容器）动态注入**，而非由对象自身直接创建或查找依赖。

1. **依赖（Dependency）**：如果一个类 `A` 的功能需要**依赖**类 `B` 才能完成，那么类 `B` 就是类 `A` 的依赖。
2. **注入（Injection）**：**创建依赖对象 `B` 的工作不是由类 `A` 自己来做**（比如在 `A` 的内部 `new B()`），而是由**外部容器（Spring）** 创建好 `B` 的实例后，通过以下方式"注入"或"传递"给 `A`：
   - **构造函数注入** (推荐)：通过类的构造函数传入
   - **Setter方法注入**：通过类的 Setter 方法传入。
   - **字段注入**

方式	实现形式	特点
构造器注入	通过反射调用构造方法创建对象，并传入依赖参数。	依赖不可变，对象初始化即完整。
Setter 注入	通过反射调用 Setter 方法为对象属性赋值。	灵活性高，允许动态更新依赖。
字段注入	通过反射直接修改对象的字段值（即使字段是 `private`）。	代码简洁，但破坏封装性，依赖关系不透明。

三种方式对比

- **Setter 注入**：通过 Setter 方法注入依赖对象
- **字段注入**：直接通过字段的 `@Autowired` 注解注入
  - @Autowired自动从 Spring 容器中注入 Bean，实现依赖管理和解耦。

    当`@Autowired`注解用于方法时，Spring会在Bean初始化阶段自动调用该方法，并注入方法参数。
  - 特别是当使用@Autowired注解注入**接口类型**的属性时，如果有**多个实现类**，Spring是如何确定具体注入哪个Bean的：使用`@Qualifier`注解（将其标记为首选的 Bean）可以指定要注入的 Bean 的名称。
- **构造器注入**：通过构造方法注入依赖对象

反射

Spring 框架就大量运用反射来达成依赖注入，依赖注入的三种方式（构造器注入、Setter 注入、字段注入）均依赖于 **反射（Reflection）** 实现动态对象创建和属性赋值。

小红书
小红书号：49607377662

在 Spring 里，容器会在运行时借助反射创建对象实例，并且将依赖的对象注入到目标对象中。例如，当你在配置文件或者使用注解声明一个 Bean 时，Spring 容器会通过反射机制来实例化这个 Bean，同时把它所依赖的其他 Bean 注入进去。

Spring Bean

定义Bean方式

- **XML 配置**（传统方式）：通过 XML 文件显式声明 Bean

```
<bean id="userService" class="com.example.UserService"/>
```

- **注解+扫描**（推荐）：通过在类上添加这些注解 `@Component`（及 `@Service`、`@Repository`、`@Controller`）。然后**启用组件扫描（component scanning）** Spring会自动检测并注册这些类为 Spring Bean
  - **衍生注解**：
    - `@Service`：业务层
    - `@Repository`：数据访问层
    - `@Controller`：Web 控制层
  - 扫描注解定义bean的前提是：**需要先配置扫描路径**
    - **自定义扫描路径**：通过 `@ComponentScan` 的 `basePackages` 参数指定：
    - **默认行为**：Spring Boot 的主启动类默认扫描其所在包及其子包。

```
@Component // 或 @Service、@Repository、@Controller
public class UserServiceImpl implements UserService {
    @Autowired
    private UserDao userDao;
}

@Configuration
@ComponentScan("com.example") // 启用组件扫描
public class AppConfig {}
```

- **Java 配置类**：用 `@Configuration` 和 `@Bean` 注解。在配置类中，定义一个方法，返回一个对象，并用@Bean注解标记，这样这个方法返回的对象就会被Spring容器管理

> `@Component` 系列注解创建bean完全交给spring容器来完成，JavaConfig的方式定义bean。它可以看做spring的“配置文件”，通过编码java代码的方式创建bean。

```
@Configuration
public class AppConfig {
    @Bean
    public UserService userService() {
        return new UserService();
    }
}
```

- 其他高级方式：如FactoryBean，动态注册等

小红书
小红书号：49607377662

条件化注册（如 Spring Boot 自动配置）

使用 `@Conditional` 系列注解按条件注册 Bean。

```
@Configuration
public class ConditionalConfig {
    @Bean
    @ConditionalOnClass(name = "com.example.SomeClass")
    public SomeBean someBean() {
        return new SomeBean();
    }
}
```

小结

现代 Spring Boot 应用通常以 **注解扫描** 和 **Java 配置类** 为主，结合 **条件化配置** 实现自动化。

Bean 加载到 IoC 容器的流程

在项目中，Spring 通过**注解扫描** 加载 Bean 的核心流程如下：

1. **扫描阶段**：Spring Boot 从 **主启动类所在包及其子包** 开始扫描，容器扫描 `@Component` 注解的类，解析为 `BeanDefinition`。
2. **实例化**：通过反射调用构造方法创建 Bean 实例。
3. **依赖注入**：填充 `@Autowired` 标记的属性和方法。
4. **初始化**：
   - 调用 `@PostConstruct` 方法。
   - 实现 `InitializingBean` 接口的 `afterPropertiesSet()`。
5. **AOP 代理**：如果需要，生成代理对象（JDK 动态代理或 CGLIB）。
6. **存入容器**：Bean 被放入 `DefaultSingletonBeanRegistry` 的单例缓存。

Bean的作用域

Spring Bean 的作用域决定了 Bean 实例的创建方式和生命周期。以下是常见作用域及其特点：

作用域	描述	适用场景
singleton（默认）	每个 Spring IoC 容器中仅存在一个 Bean 实例，所有请求返回同一实例。	无状态服务（如工具类、配置类）
prototype	每次请求（`getBean()` 或依赖注入）时创建一个新实例。	有状态对象（如用户会话、请求处理器）
request	每个 HTTP 请求创建一个新实例，仅在 Web 应用中有效	HTTP 请求相关的数据（如表单提交对象）

小红书号:49607377662

作用域	描述	适用场景
	用中有效。	（如表单提交对象）
session	每个 HTTP 会话创建一个新实例，仅在 Web 应用中有效。	用户会话数据（如购物车）
global-session	全局 HTTP 会话（Portlet 应用中使用），现已被废弃，推荐使用 `session`。	旧版 Portlet 应用（现代应用极少使用）

Spring Bean 单例的实现

Java中，单例模式通常通过私有构造函数、静态变量和静态方法来实现，但Spring管理的Bean是通过**Spring容器自身负责管理Bean的生命周期和作用域。**

Spring的Bean默认以单例（Singleton）作用域存在，Spring IoC容器会确保每个Bean定义对应一个实例。

- **实现原理**：容器通过 **三级缓存机制** 和 **单例注册表** 确保每个 Bean 定义对应唯一实例
  - Spring 容器内部维护一个单例注册表（`singletonObjects`）管理所有单例Bean实例，首次创建 Bean 后存入，后续请求直接返回该实例。
  - Spring 使用**三级缓存管理** Bean 的创建和依赖注入，解决 **循环依赖** 并确保单例唯一性

Bean的生命周期

Bean 代指的就是那些被 IoC 容器所管理的对象

1. **bean实例化：**bean本质上就是对象，Spring 容器创建对象的实例。
   - 实例化bean的三种方式，构造方法(常用),静态工厂和实例工厂
2. **属性赋值：**为 Bean 设置相关属性和依赖。
   - 例如`@Autowired`等注解注入的对象、`@Value`注入的值、`setter`方法或构造函数注入依赖和值、`@Resource`注入的各种资源。
3. **初始化：**调用对象中定义的初始化方法，执行初始化操作。
   - 这可以通过实现 **InitializingBean 接口**或在 XML 配置文件中指定 init-method 来实现。
4. **使用bean**：Bean 准备就绪，可被应用程序使用。
5. **销毁：**当 Spring 容器关闭前，销毁bean
   - ConfigurableApplicationContext接口的close()方法
   - ConfigurableApplicationContext接口的registerShutdownHook()方法

循环依赖问题

循环依赖指的是两个或多个Bean之间相互依赖，形成一个闭环。例如，Bean A依赖Bean B，而Bean B又依赖Bean A。这种情况下，Spring容器在初始化Bean的时候会陷入死循环，导致应用启动失败。

小红书
小红书号：49607377662

- **Spring通过三级缓存机制解决了单例Bean之间通过setter或字段注入导致的循环依赖问题:Spring 在Bean还未完成初始化时，就提前将其引用暴露出去（通过三级缓存），打破了循环等待的僵局。**

三级缓存

1.一级缓存singletonObjects：存放完全初始化好的单例Bean。也就是可以直接使用的单例对象。

2.二级缓存earlySingletonObjects：存放提前暴露的Bean实例，这些Bean尚未完成属性注入和初始化。

3.三级缓存singletonFactories：**Bean的ObjectFactory工厂**。存放Bean的工厂对象，用于在发生循环依赖时，**提前暴露Bean的引用**。

当一个Bean被创建时，Spring会先将该Bean的工厂对象放入singletonFactories中。这样，在后续的依赖注入过程中，如果有其他Bean需要**引用这个正在创建的Bean**，Spring可以通过这个工厂对象获取到该Bean的早期引用，从而解决循环依赖。

Spring AOP

概念：面向切面编程。

有一些功能是横跨多个类或对象，比如说日志记录，安全性检查。可以将那些与核心业务逻辑无关的**横切关注点**模块化，并通过声明的方式定义它们如何应用到目标代码中。

AOP核心思想

- **横切关注点分离**：将日志、事务、安全等横跨多个类和对象从代码从业务逻辑中分离出来
- **动态代理**：通过**动态代理**在运行时将切面逻辑插入到目标方法中。
  - **实现原理**：基于**动态代理**。
  - 如果目标对象实现了接口，默认使用 **JDK 动态代理**。
    - 如果目标对象没有实现接口，使用 **CGLIB 字节码生成**

程序运行时，Spring容器会扫描并解析所有的切面和切点，创建代理对象，将切面逻辑添加到中间，这些对象会保存在spring 容器里，在整个程序生命周期中重复使用。当

程序执行到切点的时候，代理对象就会被调用，并执行切面逻辑。

- **Aspect（切面）**：关注点（如日志、事务等），封装逻辑的类
- **Pointcut（切点）**：目标方法，决定 AOP 作用于哪些方法
- **Advice（通知）**：具体的切面逻辑，在连接点执行的动作（如 `@Before`、`@After`）

小红书
小红书号:49607377662

- **JoinPoint（连接点）**：目标方法执行的时机
- **Weaving（织入）**：将切面逻辑应用到目标方法

Spring 框架通过 IoC 管理 Bean，通过 AOP 增强 Bean 的行为

项目AOP 统一处理日志

我们项目中使用Spring AOP实现日志的统一处理。 定义一个日志记录的切面（Aspect），定义一个切点（Pointcut），表示切面逻辑用于哪个位置，随后定义日志记录逻辑。切面类使用**@Aspect， 切点@Pointcut， @Before， @after这种注解表示切面逻辑的具体位置**

有一个名为UserService的用户服务类，其中有一个方法名为getUserInfo()。假设我们希望在执行getUserInfo()方法之前打印一句话。使用Spring AOP，我们可以创建一个切面类，其中定义了**@Before通知注解修饰的before()方法**，该方法包含了切面逻辑。当调用userService.getUserInfo()方法时，实际执行的是userService的代理对象。代理对象会在执行getUserInfo()方法之前，先执行切面逻辑，即before()方法。然后再执行getUserInfo()方法的业务逻辑。

增强类型

AOP的增强类型包括@Before、@After、@AfterReturning、@AfterThrowing和@Around。

而全局异常处理通常使用@ControllerAdvice配合@ExceptionHandler注解，这样可以集中处理Controller层的异常，返回统一的错误响应。

AOP增强类型

增强类型	执行时机	示例代码
`@Before`	目标方法执行前	`@Before("pointcut()")`
`@After`	目标方法执行后（无论是否异常）	`@After("pointcut()")`
`@AfterReturning`	目标方法正常返回后	`@AfterReturning(pointcut="pointcut()", returning="result")`
`@AfterThrowing`	目标方法抛出异常后	`@AfterThrowing(pointcut="pointcut()", throwing="ex")`
`@Around`	包裹目标方法（最灵	`@Around("pointcut()")`

小红书
小红书号:49607377662

增强类型	执行时机	示例代码

全局异常处理

通过`@ControllerAdvice`和`@ExceptionHandler`统一处理异常：

AOP动态代理机制

Spring AOP就是**基于代理模式**来实现的。它的作用在**不修改原始类的情况**下**对方法调用进行拦截和增强。**

- **如果目标类 实现了接口**，**默认**使用 **JDK 动态代理。**
  - Service层接口的**事务管理**：
    - 为Service层创建代理对象。
    - 代理对象拦截方法调用并管理事务的提交和回滚。
- **如果目标类** 未实现接口，采用 **CGLIB 动态代理。**
  - Controller层的**日志切面**： **Controller 类一般不实现接口**，使用 **CGLIB 代理**

Java动态代理

1. **JDK 动态代理**
   - 原理：基于 `java.lang.reflect.Proxy` 和 `InvocationHandler`，在运行时为接口生成代理类。
   - 特点：
     - 只能代理 **接口。**
     - 生成的代理类更轻量，性能较好（尤其是 JDK 1.8 之后）。
2. **CGLIB 动态代理**
   - 原理：基于 ASM 字节码生成库，在运行时为目标类生成一个子类，并在子类中重写目标方法实现代理。
   - 特点：
     - 可以代理 **没有接口的类。**
     - 目标类和方法不能是 `final`。
     - 性能稍逊，但差距在现代 JVM 下不大。

小红书
小红书号：49607377662

Spring 事务管理

事务管理的实现主要依赖于**Spring AOP（面向切面编程）**模块，通过**动态代理机制**将事务管理逻辑加入目标方法。

典型的事务管理流程：

1. 当调用标注了@Transactional的方法时
2. Spring AOP 通过事务拦截器（TransactionInterceptor）拦截调用带 `@Transactional` 注解的方法

   > **@Transactional注解就是一个切点，定义了哪些方法需要被事务管理。**

3. **使用动态代理**自动生成代理对象，在目标方法周围添加事务管理的逻辑（开启→执行→提交/回滚）

事务注解@Transactional

当我们在方法上添加@Transactional注解时，Spring会为这个类生成一个代理对象。

代理对象会在调用目标方法之前和之后加入事务管理的逻辑，比如开启事务、提交事务或回滚事务。

自调用问题

如果一个类中的方法A调用了同一个类中的方法B，而方法B上有@Transactional注解，这时候事务会生效吗？

如果Service类中的一个方法调用另一个被@Transactional注解的方法，由于自调用的时候调用的是**目标对象的方法**而不是**代理对象的方法**，事务注解可能不会生效。这是因为自调用绕过了代理。

- 当外部调用 `MyService` 的方法时，实际调用的是 **代理对象**（如 `MyServiceProxy`）
- 但 **内部方法调用**（如 `methodA()` 调用 `methodB()`）是直接调用 **原始对象** 的方法，绕过了代理对象

需要通过获取代理对象来调用方法B，或者将方法B放到另一个类中。

事务的传播行为

- **事务代理的生成**：Spring通过AOP代理（JDK动态代理或CGLIB）来拦截带有@Transactional注解的方法。代理在调用方法前会触发事务处理逻辑。
- **事务拦截器的处理流程**：在TransactionInterceptor中，如何解析@Transactional的属性，包括传播行为，然后决定是否创建新事务、加入现有事务或挂起当前事务。
- 当一个事务方法调用另一个事务方法时，Spring需要根据传播行为来决定如何处理事务。
  - （1）**PROPAGATION_REQUIRED（默认）**：若当前存在事务，则加入；否则新建事务。

小红书
小红书号：49607377662

- **（2）PROPAGATION_REQUIRES_NEW**：挂起当前事务（若有），新建独立事务，新事务提交/回滚不影响原事务。
- **（3）PROPAGATION_NESTED**：在当前事务中创建嵌套事务（使用保存点 Savepoint），子事务回滚不影响主事务。
- **（4）PROPAGATION_MANDATORY**：强制必须在事务中调用，否则抛出 `IllegalTransactionStateException`

这些传播行为的处理应该是在代理对象的InvocationHandler中实现的，根据当前的事务状态来决定如何管理事务。

Spring 常用注解

注解	用途
`@Component`	通用 Bean 声明（需包扫描）。
`@Service`	业务层 Bean 声明。
`@Repository`	数据层 Bean 声明（自动处理 DAO 异常）。
`@Controller`	MVC 控制器声明。
`@Autowired`	自动注入依赖（按类型匹配）。
`@Qualifier`	按名称指定注入的 Bean。
`@RequestMapping`	映射 HTTP 请求到方法（如 `@GetMapping`）。
`@Transactional`	声明事务（可配置隔离级别、传播行为）。
`@Configuration`	声明配置类（结合 `@Bean` 定义 Bean）。

---

5. Spring 事务传播行为

传播行为	说明
REQUIRED	支持当前事务，不存在则新建事务（默认）。
REQUIRES_NEW	新建事务，挂起当前事务（独立提交/回滚）。
NESTED	在当前事务中嵌套子事务（可独立回滚子事务，依赖主事务提交）。
SUPPORTS	支持当前事务，不存在则以非事务执行。
NOT_SUPPORTED	非事务执行，挂起当前事务。
MANDATORY	必须存在当前事务，否则抛出异常。
NEVER	必须不存在事务，否则抛出异常。

小红书
小红书号:49607377662

示例：

- 方法 A（REQUIRED）调用方法 B（REQUIRES_NEW）：
  - B 失败回滚不影响 A；A 失败回滚不影响 B。

Spring 中的设计模式

工厂模式：BeanFactory

代理模式：AOP 动态代理

模板方法：JdbcTemplate

设计模式在框架中的应用

可以从 Spring、MyBatis、JDK 都常用的几个设计模式讲起：

设计模式	框架源码中的应用
单例模式	Spring Bean 默认单例
工厂模式	Spring 中的 `BeanFactory` / `FactoryBean`
代理模式	Spring AOP、动态代理实现事务/安全
观察者模式	Spring 事件发布机制
策略模式	SpringMVC 的 HandlerMapping、视图解析器
模板方法	JdbcTemplate / RedisTemplate
建造者模式	MyBatis XML 解析生成 Configuration 对象

设计模式 -源码应用考查
1.举例说明设计模式在某个框架源码中的应用.
2.介绍一下代理模式，说一下静态代理和动态代理的区别?
3.简述 Spring Bean 是如何实现单例模式的?
4.举例建造者模式在框架源码中的应用?

小红书
小红书号：49607377662

策略模式

策略模式则是关于对象的行为，允许定义一系列算法，并将每个算法封装在独立的类中，允许客户端在运行时选择具体的算法或行为。

策略模式的结构：

- `Context`（上下文）：用来存储一个策略对象，并利用这个策略对象来执行相应的算法。
- `Strategy`（策略接口）：定义了所有支持的算法的方法，通常是一个接口或抽象类。
- `ConcreteStrategy`（具体策略）：实现具体的算法或行为。

举例

在电商系统中，订单处理可能需要不同的折扣策略，比如新用户折扣、会员折扣、活动折扣等。使用策略模式的话，每个折扣都可以作为一个具体策略类，实现同一个策略接口。上下文类可能是订单处理类，根据用户类型或者促销活动选择合适的折扣策略进行计算。

`SpringSecurity` 认证策略

Spring Security 的认证系统深度应用了策略模式，通过 `AuthenticationProvider` **策略接口**支持多种**具体认证策略**，比如表单登录、JWT、OAuth2 等多种认证方式

每个Provider处理特定类型的认证请求。例如，`AuthenticationProvider_1` 处理用户名密码，`AuthenticationProvider_2` 处理JWT令牌等。

模板方法模式

在父类中定义**算法框架**，子类实现具体逻辑，允许子类在不改变算法结构的情况下，重新定义该算法的某些特定步骤。

模板方法模式的结构：

- `AbstractClass`（抽象类）：定义了一个模板方法，包含了算法的骨架，并调用一些**抽象方法**来实现可变的部分。
- `ConcreteClass`（具体类）：继承 `AbstractClass`，并实现其中的**抽象方法**来完成具体的操作。

`JdbcTemplate` 封装数据库访问

JdbcTemplate是Spring框架中用于简化JDBC操作的类

传统JDBC操作中，需要手动处理大量重复代码,

`JdbcTemplate` 在数据库操作中封装资源管理、异常处理等通用逻辑，比如获取连接、执行SQL、处理异常、释放资源等，剩下只要通过回调接口（如ConnectionCallback）实现具体自定义的SQL操作。

> 这里的回调接口相当于模板方法中的抽象方法去实现具体操作。

小红书
小红书号：49607377662

操作步骤	传统JDBC	Spring JdbcTemplate
获取连接	手动获取	将获取连接、异常处理、资源释放等代码封装在模板中,自动处理
异常处理	需要手动捕获并处理	将 `SQLException` 转换为 Spring 的 `DataAccessException`
资源释放	必须手动关闭	自动通过try-with-resources管理
事务控制	需要手动提交/回滚	通过声明式事务 `@Transactional` 注解管理

代理模式

代理模式为其他对象提供一种代理以控制对这个对象的访问。

允许在不修改目标对象的情况下**增强方法功能**（如日志、权限校验、事务管理等）

静态代理

手动编写代理类，代理类和目标类都需要实现相同的接口。

例子：银行账户

假设 `BankAccount` 提供 存款和 取款方法。`RealBankAccount` **目标类**是具体的银行账户，实现了 `BankAccount` 接口。

在 `BankAccountProxy` **代理类**中，增强 **日志功能**，在每次存款和取款前后打印日志

动态代理

- **基于接口的代理（JDK动态代理）**：**基于接口实现**运行时动态生成代理类
  - **在运行时**由 JVM **自动生成代理类**，然后用这个代理类去调用目标对象的方法，同时可以在方法执行前后添加额外逻辑（如日志、事务等）。
    - `Proxy` 生成代理类：**JVM 运行时动态生成代理类（字节码生成）**，代理类**实现目标接口。**
    - **通过反射调用方法**：代理对象方法调用时，自动交给 `InvocationHandler.invoke()` 通过反射调用原对象的方法，并添加增强逻辑。

- CGLIB：

小红书
小红书号:49607377662

- **基于继承**：通过**继承**方式生成**目标类的子类**实现代理
  - **不能代理 `final` 方法（无法被子类重写）**
- **无需接口**：可以直接代理普通类
- **方法拦截**：通过`MethodInterceptor`接口实现拦截代理类的所有方法调用，可以在**方法执行前后进行增强**

代理模式应用场景

- **1. Spring框架中的AOP（面向切面编程）动态代理机制**

  Spring AOP就是**基于代理模式**来实现的。它的作用是在**不修改业务代码**的情况下，给方法**添加事务、日志、权限校验等功能。**

  - **如果目标类 实现了接口**，**默认**使用 **JDK 动态代理。**
    - Service层接口的**事务管理**：
      - 为Service层创建代理对象。
      - 代理对象拦截方法调用并管理事务的提交和回滚。
    - **如果目标类** 未实现接口，采用 **CGLIB 动态代理。**
      - Controller层的**日志切面**： **Controller 类一般不实现接口**，使用 **CGLIB 代理**

  2. 远程方法调用（RPC）中的使用

  希望客户端可以像**调用本地方法一样调用远程服务。**但本地代码**不能直接访问远程服务**，因此必须使用 **代理模式。**

常用注解

在 **Spring Boot** 和 **MyBatis** 项目中，常用的注解包括以下几类：

---

1. Spring 相关注解

(1) 常见的 Spring 组件注解

- `@Component`：通用组件，受 Spring 容器管理。
- `@Service`：标识 **业务层** Bean。
- `@Controller`：标识 **控制层** Bean，配合 `@RequestMapping` 处理 HTTP 请求。
- `@RestController`：`@Controller + @ResponseBody`，返回 JSON 格式数据。

(2) 依赖注入

小红书
小红书号：49607377662

- `@Autowired`：**按类型**自动注入 Bean（默认从 Spring 容器中查找）。
  - 当使用`@Autowired`注解对一个接口类型的属性进行注入时，Spring 会在容器中寻找实现了该接口的 Bean，如果存在多个实现类，Spring 会依据特定的规则来确定具体注入哪个 Bean。
- `@Qualifier("beanName")`：指定 Bean 的名称进行注入。
- `@Resource(name="beanName")`：JDK 提供的依赖注入，默认按名称匹配。
- `@Value("${config.value}")`：从 `application.properties` 读取值。

(3) 配置相关

- `@Configuration`：标识 **配置类**，用于定义 Bean。
- `@Bean`：手动注册 Bean。
- `@ComponentScan("com.example")`：指定 Spring 组件扫描路径。

(4) AOP 相关

- `@Aspect`：声明 **切面**。
- `@Before`、`@After`、`@Around`：定义 **AOP 切面逻辑**。

---

2. Spring MVC 相关注解

- `@RequestMapping("/path")`：映射 HTTP 请求。
- `@GetMapping`、`@PostMapping`、`@PutMapping`、`@DeleteMapping`：RESTful 风格 API 请求映射。
- `@RequestParam("param")`：获取 URL 请求参数。
- `@PathVariable("id")`：获取路径参数，例如 `/user/{id}`。
- `@RequestBody`：接收 JSON 格式的请求体参数。

---

3. Spring Boot 相关

- `@SpringBootApplication`：Spring Boot 入口类（包含 `@Configuration`、`@EnableAutoConfiguration`、`@ComponentScan`）。
- `@EnableCaching`：开启缓存。
- `@EnableScheduling`：开启定时任务支持。

---

4. MyBatis / MyBatis-Plus 相关

- `@Mapper`：标识 **MyBatis 持久层接口**。
- `@Select("SQL语句")`、`@Insert`、`@Update`、`@Delete`：MyBatis 注解式 SQL。
- `@TableName("table_name")`：MyBatis-Plus 指定数据库表。
- `@TableId(type = IdType.AUTO)`：主键策略。

---

5. 事务管理

- `@Transactional`：**事务管理**，可用于 **方法级别** 或 **类级别**。

小红书
小红书号：49607377662

@Transactional

- **保证原子性**：事务中的所有 SQL **要么全部成功，要么全部回滚。**
- **保证一致性**：如果业务逻辑中某个 SQL 执行失败，整个事务都会回滚。
- **提升性能**：减少数据库的**自动提交（auto commit）**次数，提高执行效率。

Q1: @Transactional 为什么只能作用在 `public` 方法上?

- 因为 Spring 事务基于 **AOP 代理机制**，只有 `public` 方法才能被代理。

Q2: 为什么 @Transactional 不能用于 `final` 和 `static` 方法?

- 因为 Spring 使用 **动态代理（JDK 代理 / CGLIB 代理）**来管理事务，而 `final` 方法不能被子类重写，无法被代理。

Q3: @Transactional 事务是如何回滚的?

- 默认回滚 `RuntimeException` **和** `Error`，不会回滚 `CheckedException`。
- 需要手动指定 `rollbackFor = Exception.class` 来回滚所有异常。

Q4: 什么是事务的传播机制?

- 事务传播机制决定了**方法 A 调用方法 B 时，B 是否复用 A 的事务。**
- **常见机制**：
  - `REQUIRED`（默认）：如果有事务，就加入；否则新建事务。
  - `REQUIRES_NEW`：新建事务，原事务挂起。
  - `NESTED`：嵌套事务，子事务回滚不影响主事务。

Q5: 事务失效的常见原因?

1. **方法不是 `public`**
2. **同类方法调用**
3. **多线程 / `@Async` 方法**
4. **事务传播机制问题**

小红书
小红书号：49607377662